



Linear regression

```
In [ ]: #1-6
# --- Step 1: Import Libraries ---
import numpy as np
import pandas as pd

# --- Step 2: Load Dataset ---
df = pd.read_csv("advertising.csv") # Change file name if needed
print("✅ Dataset Loaded Successfully\n")
print(df.head())

# --- Step 3: Handle Missing Values ---
print("\n--- Dataset Info ---")
print(df.info())

print("\n--- Missing Values ---")
print(df.isnull().sum())
df = df.dropna()

# --- Step 4: Define Linear Regression using Gradient Descent ---
def linear_regression_gd(df, features, target, lr=0.01, epochs=1000):
    X = df[features].values
    y = df[target].values.reshape(-1, 1)

    # Add bias term (x0 = 1)
    X_b = np.c_[np.ones((X.shape[0], 1)), X]
    n_samples, n_features = X_b.shape

    # Initialize theta randomly
    theta = np.random.randn(n_features, 1)

    # Gradient Descent
    for i in range(epochs):
        # Predictions
        y_pred = X_b @ theta

        # Compute gradient
        gradients = (2 / n_samples) * X_b.T @ (y_pred - y)

        # Update parameters
        theta -= lr * gradients

    # Final prediction
    y_pred = X_b @ theta
    rmse = np.sqrt(np.mean((y - y_pred) ** 2))

    # Display coefficients
    print("\nModel Coefficients:")
    for i, f in enumerate(["Intercept"] + features):
        print(f"{f}: {theta[i][0]:.4f}")
```

```

print(f"\n📊 Final RMSE: {rmse:.3f}")

# --- Step 5: Call Function ---
features = ["Radio", "Newspaper"] # Try also ["TV", "Radio"]
target = "Sales"

linear_regression_gd(df, features, target, lr=0.0005, epochs=2000)

```

✅ Dataset Loaded Successfully

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

```

--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   TV          200 non-null   float64
1   Radio       200 non-null   float64
2   Newspaper   200 non-null   float64
3   Sales       200 non-null   float64
dtypes: float64(4)
memory usage: 6.4 KB
None

```

```

--- Missing Values ---
TV          0
Radio       0
Newspaper   0
Sales       0
dtype: int64

```

```

Model Coefficients:
Intercept: 12.0607
Radio: 0.1195
Newspaper: 0.0095

```

📊 RMSE: 4.934

In []: #7-18

```

# --- Step 1: Import Libraries ---
import numpy as np
import pandas as pd

# --- Step 2: Load Dataset ---
df = pd.read_csv("CAR DETAILS FROM CAR DEKHO.csv") # Change file name as needed
print("✅ Dataset Loaded Successfully\n")

```

```

print(df.head())

# --- Step 3: Encode categorical columns (example) ---
# Encode transmission (Manual = 0, Automatic = 1)
if 'transmission' in df.columns:
    df['transmission_enc'] = df['transmission'].apply(lambda x: 1 if 'Auto' in x else 0)

# Encode owner numerically (First Owner = 1, Second Owner = 2, etc.)
if 'owner' in df.columns:
    def encode_owner(x):
        if 'First' in x: return 1
        elif 'Second' in x: return 2
        elif 'Third' in x: return 3
        else: return 4
    df['owner_num'] = df['owner'].apply(encode_owner)

# --- Step 4: Understand and Cleanup (if required) ---
print("\n--- Dataset Info ---")
print(df.info())

print("\n--- Missing Values ---")
print(df.isnull().sum())
df = df.dropna()

# --- Step 5: Define Linear Regression using Gradient Descent ---
def linear_regression_gd(df, features, target, lr=0.0000001, epochs=1000):
    X = df[features].values
    y = df[target].values.reshape(-1, 1)

    # Add bias term (x0 = 1)
    X_b = np.c_[np.ones((X.shape[0], 1)), X]
    n_samples, n_features = X_b.shape

    # Initialize parameters randomly
    theta = np.random.randn(n_features, 1)

    # Gradient Descent
    for i in range(epochs):
        y_pred = X_b @ theta
        gradients = (2 / n_samples) * X_b.T @ (y_pred - y)
        theta -= lr * gradients

    # Final predictions
    y_pred = X_b @ theta
    rmse = np.sqrt(np.mean((y - y_pred) ** 2))

    # Display coefficients
    print("\nModel Coefficients:")
    for i, f in enumerate(["Intercept"] + features):
        print(f"{f}: {theta[i][0]:.4f}")

    print(f"\nFinal RMSE: {rmse:.3f}")

```

```
# --- Step 6: Call Function ---  
features = ["owner_num", "year"] # Try adding more like ["owner_num", "year"  
target = "selling_price"  
  
linear_regression_gd(df, features, target, lr=0.0000001, epochs=2000)
```

✓ Dataset Loaded Successfully

	name	year	selling_price	km_driven	fuel	\
0	Maruti 800 AC	2007	60000	70000	Petrol	
1	Maruti Wagon R LXi Minor	2007	135000	50000	Petrol	
2	Hyundai Verna 1.6 SX	2012	600000	100000	Diesel	
3	Datsun RediGO T Option	2017	250000	46000	Petrol	
4	Honda Amaze VX i-DTEC	2014	450000	141000	Diesel	

	seller_type	transmission	owner
0	Individual	Manual	First Owner
1	Individual	Manual	First Owner
2	Individual	Manual	First Owner
3	Individual	Manual	First Owner
4	Individual	Manual	Second Owner

--- Dataset Info ---

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 4340 entries, 0 to 4339

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	name	4340 non-null	object
1	year	4340 non-null	int64
2	selling_price	4340 non-null	int64
3	km_driven	4340 non-null	int64
4	fuel	4340 non-null	object
5	seller_type	4340 non-null	object
6	transmission	4340 non-null	object
7	owner	4340 non-null	object
8	transmission_enc	4340 non-null	int64
9	owner_num	4340 non-null	int64

dtypes: int64(5), object(5)

memory usage: 339.2+ KB

None

--- Missing Values ---

name	0
year	0
selling_price	0
km_driven	0
fuel	0
seller_type	0
transmission	0
owner	0
transmission_enc	0
owner_num	0

dtype: int64

Model Coefficients:

Intercept: -109419464.9385

owner_num: -29712.9196

year: 54625.9778

 RMSE: 526240.402

SVM

In []: #19-26

```
# --- Step 1: Import Libraries ---
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,

# --- Step 2: Load Dataset ---
# Replace 'your_dataset.csv' with your file name
df = pd.read_csv("Social_Network_Ads.csv")

# --- Step 3: Understand the Dataset ---
print("First 5 rows:\n", df.head())
print("\nInfo:\n", df.info())
print("\nMissing values:\n", df.isnull().sum())

# --- Step 4: Data Cleanup (if required) ---
# Drop duplicates
df = df.drop_duplicates()

# Handle missing values (if any)
df = df.dropna()

# --- Step 5: Encode Categorical Columns ---
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object': # Encode categorical columns
        df[col] = le.fit_transform(df[col])

# --- Step 6: Split Dataset into Features and Target ---
# Replace 'Purchased' with your actual target column name
X = df.drop('Purchased', axis=1)
y = df['Purchased']

# --- Step 7: Split into Train & Test Sets ---
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# --- Step 8: Feature Scaling ---
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# --- Step 9: Train SVM Model (Linear Kernel) ---
model = SVC(kernel='linear', random_state=42) #for RBF but rbf instead of linear
model.fit(X_train, y_train)
```

```
# --- Step 10: Make Predictions ---
y_pred = model.predict(X_test)

# --- Step 11: Evaluate Model ---
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

# --- Step 12: Display Results ---
print("\nConfusion Matrix:\n", cm)
print("\nModel Evaluation Metrics:")
print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall: {rec:.4f}")
print(f"F1 Score: {f1:.4f}")
```

First 5 rows:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 400 entries, 0 to 399

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	User ID	400 non-null	int64
1	Gender	400 non-null	object
2	Age	400 non-null	int64
3	EstimatedSalary	400 non-null	int64
4	Purchased	400 non-null	int64

dtypes: int64(4), object(1)

memory usage: 15.8+ KB

Info:

None

Missing values:

User ID	0
Gender	0
Age	0
EstimatedSalary	0
Purchased	0

dtype: int64

Confusion Matrix:

```
[[50  2]
 [ 9 19]]
```

Model Evaluation Metrics:

Accuracy: 0.8625

Precision: 0.9048

Recall: 0.6786

F1 Score: 0.7755

```
In [ ]: #27-34
# --- Step 1: Import Libraries ---
import pandas as pd
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,

# --- Step 2: Load the Iris Dataset ---
iris = datasets.load_iris()
```



```

# Convert to DataFrame for convenience
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['species'] = iris.target

# --- Step 3: Understand the Dataset ---
print("First 5 rows:\n", df.head())
print("\nDataset Info:\n")
print(df.info())
print("\nMissing Values:\n", df.isnull().sum())

# --- Step 4: Data Cleanup (if required) ---
# Remove duplicates or missing values (if any)
df = df.drop_duplicates()
df = df.dropna()

# --- Step 5: Split Dataset into Features and Target ---
X = df.drop('species', axis=1)
y = df['species']

# --- Step 6: Split into Training and Test Sets ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# --- Step 7: Feature Scaling ---
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# --- Step 8: Train SVM Model (RBF Kernel) ---
model = SVC(kernel='rbf', random_state=42) #***** for linear put linear
model.fit(X_train, y_train)

# --- Step 9: Make Predictions ---
y_pred = model.predict(X_test)

# --- Step 10: Evaluate Model ---
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='macro')
rec = recall_score(y_test, y_pred, average='macro')
f1 = f1_score(y_test, y_pred, average='macro')

# --- Step 11: Display Results ---
print("\nConfusion Matrix:\n", cm)
print("\nModel Evaluation Metrics:")
print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall    : {rec:.4f}")
print(f"F1 Score  : {f1:.4f}")

```

First 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	species
0	0
1	0
2	0
3	0
4	0

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
3   petal width (cm)       150 non-null   float64
4   species                150 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
None
```

Missing Values:

sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
species	0

dtype: int64

Confusion Matrix:

```
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]
```

Model Evaluation Metrics:

Accuracy : 0.9667
Precision: 0.9697
Recall : 0.9667
F1 Score : 0.9666

Logistic regression

In []: *#easier version*

```

import numpy as np
from sklearn.datasets import load_iris

# --- Load Data ---
iris = load_iris()
X, y = iris.data, iris.target
X = np.c_[np.ones((X.shape[0], 1)), X]      # Add bias column
Y = np.eye(3)[y]                            # One-hot encode

# --- Initialize ---
np.random.seed(0)
theta = np.random.randn(X.shape[1], 3)
lr, epochs = 0.1, 1000

# --- Functions ---
def softmax(z):
    e = np.exp(z - z.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)

# --- Train ---
for i in range(epochs):
    y_pred = softmax(X @ theta)
    grad = X.T @ (y_pred - Y) / len(X)
    theta -= lr * grad

# --- Predict & Accuracy ---
pred = np.argmax(softmax(X @ theta), axis=1)
acc = np.mean(pred == y)
print(f"Accuracy: {acc*100:.2f}%")

```

Accuracy: 98.00%

```

In [ ]: #38
# --- Step 1: Import Libraries ---
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris

# --- Step 2: Load Dataset ---
iris = load_iris()
X = iris.data      # features
y = iris.target    # labels (0, 1, 2)
n_samples, n_features = X.shape
n_classes = len(np.unique(y))

# --- Step 3: One-hot Encode y ---
Y = np.zeros((n_samples, n_classes))
for i in range(n_samples):
    Y[i, y[i]] = 1

# --- Step 4: Add Bias Term ---
X_b = np.c_[np.ones((n_samples, 1)), X]

```

```

# --- Step 5: Initialize Parameters ---
np.random.seed(42)
theta = np.random.randn(X_b.shape[1], n_classes)

# --- Step 6: Define Softmax Function ---
def softmax(z):
    exp_z = np.exp(z - np.max(z, axis=1, keepdims=True))
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)

# --- Step 7: Define Cross-Entropy Loss ---
def compute_loss(Y, y_pred):
    return -np.mean(np.sum(Y * np.log(y_pred + 1e-9), axis=1))

# --- Step 8: Training using Gradient Descent ---
lr = 0.1
epochs = 1000

for i in range(epochs):
    z = X_b @ theta
    y_pred = softmax(z)
    loss = compute_loss(Y, y_pred)

    # Gradient
    grad = X_b.T @ (y_pred - Y) / n_samples
    theta -= lr * grad

    if i % 200 == 0:
        print(f"Epoch {i}, Loss: {loss:.4f}")

# --- Step 9: Predictions ---
y_pred_class = np.argmax(softmax(X_b @ theta), axis=1)

# --- Step 10: Evaluate Accuracy ---
accuracy = np.mean(y_pred_class == y)
print(f"\nModel Accuracy: {accuracy*100:.2f}%")

```

```

Epoch 0, Loss: 13.4290
Epoch 200, Loss: 0.2659
Epoch 400, Loss: 0.1929
Epoch 600, Loss: 0.1596
Epoch 800, Loss: 0.1401

```

Model Accuracy: 98.00%

Decision Tree

```

In [ ]: #35-37
# --- Step 1: Import Libraries ---
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_r

```

```

# --- Step 2: Load Dataset ---
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['target'] = iris.target

# --- Step 3: Understand the Dataset ---
print("First 5 rows:\n", df.head())
print("\nInfo:\n", df.info())
print("\nMissing values:\n", df.isnull().sum())

# --- Step 4: Data Cleanup (if required) ---
df = df.drop_duplicates()
df = df.dropna()

# --- Step 5: Split Features and Target ---
X = df.drop('target', axis=1)
y = df['target']

# --- Step 6: Split into Train & Test Sets ---
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# --- Step 7: Build Decision Tree Model ---
# Change the criterion here → "gini" | "entropy" | "log_loss"
model = DecisionTreeClassifier(criterion='gini', max_depth=4, min_samples_split=2)

# --- Step 8: Train the Model ---
model.fit(X_train, y_train)

# --- Step 9: Make Predictions ---
y_pred = model.predict(X_test)

# --- Step 10: Evaluate Model ---
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# --- Step 11: Display Results ---
print("\nConfusion Matrix:\n", cm)
print("\nAccuracy:", round(acc, 4))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

```

First 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	target
0	0
1	0
2	0
3	0
4	0

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 150 entries, 0 to 149

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	target	150 non-null	int64

dtypes: float64(4), int64(1)

memory usage: 6.0 KB

Info:

None

Missing values:

sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
target	0

dtype: int64

Confusion Matrix:

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30

weighted avg 1.00 1.00 1.00 30

Ensemble Models

```
In [ ]: #39-42
# --- Step 1: Import Libraries ---
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_r

# Ensemble Models
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier, GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier

# --- Step 2: Load Dataset ---
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['target'] = iris.target

# --- Step 3: Understand the Dataset ---
print("First 5 rows:\n", df.head())
print("\nInfo:\n", df.info())
print("\nMissing values:\n", df.isnull().sum())

# --- Step 4: Data Cleanup (if required) ---
df = df.drop_duplicates()
df = df.dropna()

# --- Step 5: Split Features and Target ---
X = df.drop('target', axis=1)
y = df['target']

# --- Step 6: Split into Train & Test Sets ---
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# --- Step 7: Build the Model ---
# Uncomment any ONE model below 📌

# 1📌 Bagging Classifier
# model = BaggingClassifier(estimator=DecisionTreeClassifier(), n_estimators=100, random_state=42)

# 2📌 Random Forest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)

# 3📌 Gradient Boosting Classifier
# model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, random_state=42)

# 4📌 AdaBoost Classifier
# model = AdaBoostClassifier(n_estimators=50, learning_rate=1.0, random_state=42)

# --- Step 8: Train the Model ---
```

```
model.fit(X_train, y_train)

# --- Step 9: Make Predictions ---
y_pred = model.predict(X_test)

# --- Step 10: Evaluate Model ---
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# --- Step 11: Display Results ---
print("\nConfusion Matrix:\n", cm)
print("\nAccuracy:", round(acc, 4))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```


First 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	target
0	0
1	0
2	0
3	0
4	0

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 150 entries, 0 to 149

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	target	150 non-null	int64

dtypes: float64(4), int64(1)

memory usage: 6.0 KB

Info:

None

Missing values:

sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
target	0

dtype: int64

Confusion Matrix:

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30

weighted avg	1.00	1.00	1.00	30
--------------	------	------	------	----

PCA

```
In [ ]: #43
# --- Step 1: Import libraries ---
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score

# --- Step 2: Load the dataset ---
iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
df['target'] = iris.target

X = df.drop('target', axis=1)
y = df['target']

print("Shape of X:", X.shape)
print("Classes:", iris.target_names)

# --- Step 3: Split data into train and test sets ---
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# --- Step 4: Build Random Forest Classifier (Without PCA) ---
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

acc_without_pca = accuracy_score(y_test, y_pred)
print("\nAccuracy without PCA:", acc_without_pca)

# --- Step 5: Apply PCA for Dimensionality Reduction ---
pca = PCA(n_components=2) # reduce features from 4 to 2
X_pca = pca.fit_transform(X)

# Split again after PCA
X_train_pca, X_test_pca, y_train_pca, y_test_pca = train_test_split(X_pca, y, test_size=0.3, random_state=42)

# --- Step 6: Build Random Forest Classifier (With PCA) ---
rf_pca = RandomForestClassifier(random_state=42)
rf_pca.fit(X_train_pca, y_train_pca)
y_pred_pca = rf_pca.predict(X_test_pca)

acc_with_pca = accuracy_score(y_test_pca, y_pred_pca)
print("Accuracy with PCA:", acc_with_pca)

# --- Step 7: Compare Results ---
print("\nComparison:")
```

```
print(f"Without PCA Accuracy: {acc_without_pca:.3f}")  
print(f"With PCA Accuracy: {acc_with_pca:.3f}")
```

Shape of X: (150, 4)

Classes: ['setosa' 'versicolor' 'virginica']

Accuracy without PCA: 1.0

Accuracy with PCA: 0.9777777777777777

Comparison:

Without PCA Accuracy: 1.000

With PCA Accuracy: 0.978